

Control de Acceso con Módulo RFID RC522

Sistemas Programables | Benjamín Salas Vizcarra | Septiembre 2026

1. Introducción

La tecnología RFID (Radio Frequency Identification) permite la identificación automática de objetos o personas mediante ondas de radiofrecuencia. El módulo RC522 es un lector RFID de 13.56 MHz ampliamente utilizado en sistemas de control de acceso, registro de asistencia y automatización industrial.

En esta práctica se implementó un sistema de control de acceso que distingue entre tarjetas autorizadas y no autorizadas, indicando el resultado mediante dos LEDs: verde para acceso concedido y rojo para acceso denegado. La comunicación entre el Arduino UNO R4 WiFi y el módulo RC522 se realiza mediante el protocolo SPI.

2. Objetivos

- Comprender el funcionamiento del protocolo SPI para la comunicación con el módulo RFID RC522.
- Implementar un sistema de control de acceso basado en la lectura del UID de tarjetas RFID.
- Distinguir entre tarjetas autorizadas y no autorizadas mediante indicadores visuales (LEDs).
- Utilizar millis() para el control de tiempo de los LEDs sin bloquear el programa.
- Verificar el esquema de conexiones en Tinkercad antes del montaje físico.

3. Materiales utilizados

Componente	Descripción	Cantidad
Arduino UNO R4 WiFi	Microcontrolador principal	1
Módulo RFID RC522	Lector RFID 13.56 MHz — protocolo SPI	1
Tarjeta RFID	Credencial pasiva para pruebas de acceso	1
LED verde	Indicador de acceso concedido — Pin D2	1
LED rojo	Indicador de acceso denegado — Pin D3	1
Resistencia 220 Ω	Limitadora de corriente para LEDs	2
Protoboard	Plataforma de prototipado	1
Cables Dupont	Conexiones entre Arduino y módulo	varios

4. Descripción del sistema

El sistema lee continuamente el UID de cualquier tarjeta RFID acercada al módulo RC522. Si el UID coincide con el valor almacenado en el arreglo UID_AUTORIZADO, el Arduino enciende el LED verde durante 2 segundos y reporta 'ACCESO CONCEDIDO' en el Monitor Serie. Si el UID no coincide, enciende el LED rojo durante 2 segundos y reporta 'ACCESO DENEGADO'. El control de tiempo se realiza con millis() para no bloquear el programa entre lecturas.

Mapeo de conexiones SPI (Arduino → RC522):

Pin Arduino	Pin RC522	Color cable	Función
3.3V	VCC	Rojo	Alimentación del módulo
GND	GND	Negro	Tierra común
D9	RST	Naranja	Reset del módulo
D10	SDA / SS	Blanco	Selección de esclavo SPI
D11	MOSI	Azul	Datos maestro → esclavo
D12	MISO	Verde	Datos esclavo → maestro
D13	SCK	Amarillo	Reloj del bus SPI
D2	—	—	LED verde (Acceso concedido)
D3	—	—	LED rojo (Acceso denegado)

5. Diagrama del circuito (Tinkercad)

Dado que Tinkercad no incluye el componente RC522 en su catálogo, se usó una protoboard secundaria para representar sus pines, identificada con la etiqueta 'Módulo RFID RC522 (Alimentado a 3.3V)'.

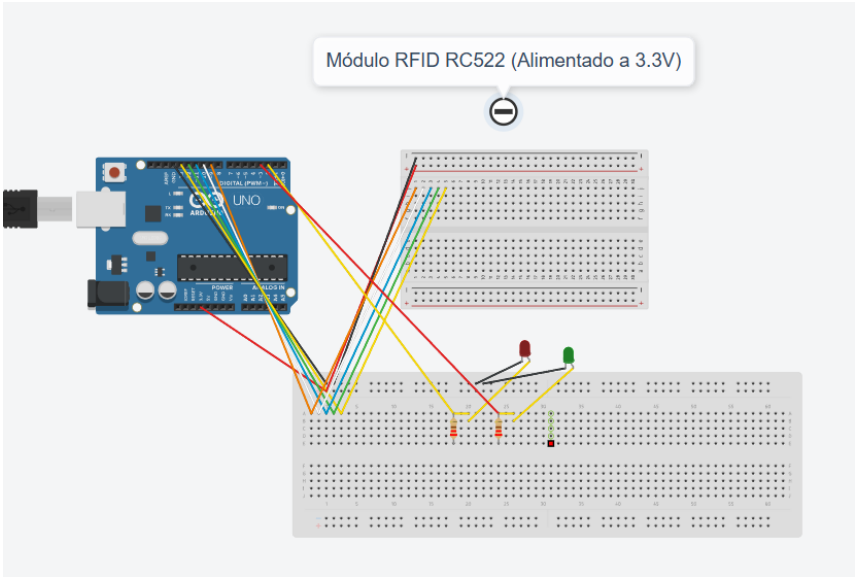


Figura 1. Diagrama de conexiones en Tinkercad con la protoboard secundaria representando el módulo RC522.

6. Evidencia del circuito físico

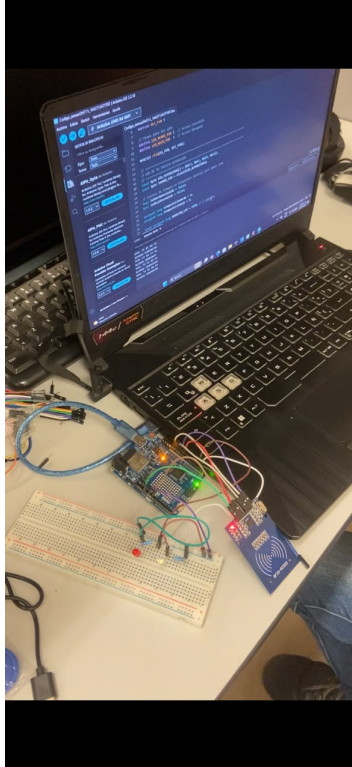


Figura 2. Montaje físico completo con Arduino UNO R4 WiFi, módulo RC522 y protoboard.

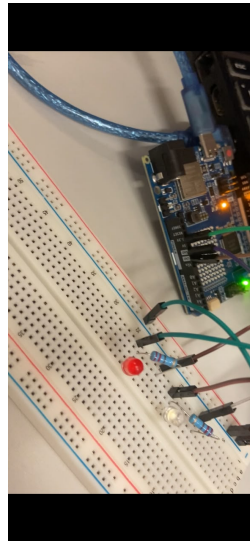


Figura 3. Detalle de los LEDs rojo y verde con resistencias de $220\ \Omega$ en la protoboard.

7. Metodología

Paso 1: Verificación en Tinkercad

Se diseñó el esquema de conexiones verificando que no existieran cortocircuitos en el bus SPI y que la alimentación de 3.3V y GND fueran correctas.

Paso 2: Instalación de la librería MFRC522

Se instaló desde el Gestor de Bibliotecas del Arduino IDE 2.x. Esta librería provee las funciones de inicialización, lectura de UID y control del módulo RC522.

Paso 3: Programación del Arduino

Se definieron los pines SS (D10) y RST (D9), los pines D2 y D3 como salidas, y el UID autorizado como arreglo de bytes. Se usó millis() para el control de tiempo de los LEDs sin bloquear el loop().

Paso 4: Montaje del circuito físico

Se conectó el módulo RC522 respetando la alimentación de 3.3V. Se colocaron los LEDs con sus resistencias y se verificó la continuidad de GND.

Paso 5: Pruebas de funcionamiento

Se realizaron pruebas con la tarjeta autorizada y tarjetas no autorizadas, verificando el encendido correcto de cada LED y los mensajes en el Monitor Serie.

8. Depuración y errores identificados

- **Pin RST duplicado:** Se había conectado un segundo cable al pin RESET del zócalo POWER. Se retiró, dejando únicamente D9 como RST.
- **Cortocircuito en protoboard:** Las líneas SPI se introdujeron en la misma columna. Se distribuyó cada señal en columnas independientes.
- **LED sin encender por GND aislado:** El riel negativo estaba desconectado del GND del Arduino. Se agregó un puente negro para cerrar el circuito.
- **Módulo alimentado a 5V:** El RC522 requiere 3.3V. Se verificó que el cable rojo fuera al pin 3.3V del Arduino.

9. Conclusiones

- El protocolo SPI permite una comunicación rápida y confiable entre el Arduino y el módulo RC522 usando cuatro líneas de datos más RST.
- La identificación por UID permite distinguir de forma única cada tarjeta RFID, siendo la base de cualquier sistema de control de acceso.
- El uso de millis() en lugar de delay() permite que el sistema permanezca siempre activo para detectar nuevas tarjetas incluso mientras el LED está encendido.
- La verificación previa en Tinkercad fue clave para identificar errores de cableado antes del montaje físico, ahorrando tiempo y evitando daños al hardware.

10. Preguntas de la práctica

Las siguientes preguntas fueron planteadas por el profesor como parte de la evaluación de la práctica:

Pregunta 1: ¿Qué significan las siglas SCK, MOSI, MISO y CS?

SCK (Serial Clock): es la señal de reloj generada por el maestro que sincroniza la transferencia de datos en el bus SPI. MOSI (Master Out Slave In): línea por donde el maestro (Arduino) envía datos al esclavo (RC522). MISO (Master In Slave Out): línea por donde el esclavo envía datos de vuelta al maestro. CS (Chip Select), también llamado SS (Slave Select): señal que el maestro activa en bajo para seleccionar con qué esclavo desea comunicarse; en el RC522 este pin se llama SDA.

Pregunta 2: ¿Por qué el pin del módulo dice 'SDA' si no es I2C? ¿Qué función cumple en realidad?

El RC522 etiqueta ese pin como 'SDA' por convención del fabricante, pero en el contexto del protocolo SPI cumple la función de CS/SS (Chip Select / Slave Select). Su función real es indicarle al módulo cuándo el maestro quiere comunicarse con él: cuando el Arduino lo pone en LOW, el RC522 sabe que debe escuchar el bus SPI.

Pregunta 3: ¿Por qué línea viaja el UID del lector al Arduino?

El UID viaja por la línea MISO (Master In Slave Out), que es el canal de datos en dirección esclavo → maestro. Cuando el Arduino solicita la lectura de una tarjeta, el RC522 responde enviando los bytes del UID a través de esta línea, sincronizados por la señal de reloj SCK.

Pregunta 4: ¿Qué pasó al desconectar MISO y por qué?

Al desconectar MISO, el Arduino dejó de recibir los datos provenientes del módulo RC522. El programa no pudo leer el UID de ninguna tarjeta porque la línea de retorno de datos estaba interrumpida. El sistema quedó sin respuesta ante cualquier tarjeta acercada, ya que sin MISO no existe canal de comunicación del esclavo hacia el maestro.

Pregunta 5: ¿Por qué no se permitió usar delay() en esta práctica? ¿Qué problema habría causado?

El uso de delay() bloquea completamente la ejecución del programa durante el tiempo indicado. Si se hubiera usado delay(2000) para mantener el LED encendido, el Arduino habría quedado inactivo por 2 segundos, sin poder detectar nuevas tarjetas ni atender ninguna otra tarea. Con millis() el LED se controla por tiempo sin bloquear el loop(), permitiendo que el sistema siempre esté listo para leer la siguiente tarjeta.

Pregunta 6: Si quisieran conectar un segundo lector RC522 al mismo Arduino, ¿qué cables compartirían y cuál tendría que ser diferente?

Los dos lectores compartirían las líneas SCK (D13), MOSI (D11), MISO (D12) y la alimentación 3.3V y GND, ya que el bus SPI es compartido entre todos los dispositivos. Lo que debe ser diferente es el pin CS/SS (SDA) de cada módulo: cada RC522 necesita su propio pin de selección en el Arduino (por ejemplo D10 para el primero y D8 para el segundo), de modo que el maestro pueda activarlos de forma independiente.

Repositorio GitHub:

<https://github.com/Benja-S-V/Universidad/tree/main/PracticaRFID>